

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3032 - Construcción de Compiladores



Proyecto No. 2

Análisis Semántico de Compiscript

Cindy Gualim, 21226

Javier Linares, 231135

Gadiel Ocaña, 231270

Jennifer Toxcon, 21276

GUATEMALA, 6 de septiembre de 2026

Índice

1. Introducción	4
2. Objetivos del proyecto	5
2.1 Objetivo general	5
2.2 Objetivos específicos	5
3. Fundamentos teóricos	5
3.1 Análisis semántico	5
3.2 Sistema de tipos	6
3.3 Tabla de símbolos	7
3.4 Ámbitos y entornos anidados	7
3.5 Listener o Visitor de ANTLR	7
3.6 Verificación de tipos	8
3.7 Detección de código muerto	8
3.8 Funciones anidadas, recursividad y closures	9
4. Diseño de componentes	10
4.1 Módulo léxico	10
4.2 Módulo gramatical	11
4.3 Módulo sintáctico	11
4.4 Módulo semántico	11
4.5 Módulo de tabla de símbolos	12
4.6 Módulo de visualización del árbol sintáctico	12
4.7 Interfaz gráfica o IDE	12
5. Diseño de estructuras de datos	13
5.1 Estructura de la tabla de símbolos	13
5.2 Representación de símbolos	14
5.3 Manejo de entornos y ámbitos	14
5.4 Estructura de los errores semánticos	15
6. Interfaz gráfica — Vistas	16
6.1 Editor de código	16
6.2 Visualización del árbol sintáctico	16
6.3 Reporte de errores	17
6.4 Visualización de la tabla de símbolos	18
7. Conjuntos de prueba	20

7.1 Sistema de tipos	21
7.2 Manejo de ámbitos	21
7.3 Funciones y procedimientos	22
7.4 Control de flujo	22
7.5 Clases y objetos	22
7.6 Listas y estructuras de datos	22
7.7 Validaciones generales	22
8. Restricciones cumplidas	23
9. Manual de instalación y ejecución	24
9.1 Requisitos y dependencias	24
9.2 Ejecución del IDE	24
9.3 Compilación de archivos .cps	25
9.4 Ejecución de las pruebas	25
10. Participación y distribución del trabajo	25
11. Conclusiones	26
12. Enlace al repositorio de GitHub	28
13. Referencias	28

1. Introducción

Este proyecto consiste en crear la parte de un compilador que comprueba si un programa escrito en Compiscript tiene sentido y cumple las reglas del lenguaje.

Compiscript es un lenguaje de programación parecido a TypeScript, que permite utilizar variables, funciones, clases, objetos, listas, condiciones y ciclos.

Cuando el sistema recibe un archivo con extensión .cps, realiza los siguientes pasos:

1. Lee y analiza el código utilizando ANTLR
2. Construye una representación interna del programa llamada árbol sintáctico
3. Recorre ese árbol para verificar que el código cumpla las reglas semánticas

Estas reglas permiten detectar errores como:

- Utilizar una variable que no existe
- Asignar un valor de un tipo incorrecto
- Llamar una función con argumentos equivocados
- Utilizar incorrectamente una clase, un objeto o una lista
- Escribir instrucciones de control de flujo en lugares donde no están permitidas

La gramática del lenguaje se encuentra en `grammar/Compiscript.g4`

El equipo la amplió para permitir números decimales mediante el tipo y el valor literal float

El análisis se realiza en dos recorridos:

- Primer recorrido: registra todas las funciones globales y clases antes de analizar el programa completo. Esto permite usar una función o heredar de una clase aunque su declaración aparezca después en el archivo
- Segundo recorrido: revisa todas las instrucciones y detecta los errores semánticos

Para controlar las variables, funciones, clases y sus tipos, se utiliza una tabla de símbolos, esta tabla la información según el lugar donde fue declarada, por ejemplo, una variable puede pertenecer al programa completo, a una función, a una clase o únicamente a un bloque de código

El proyecto también incluye un IDE desarrollado con CustomTkinter, donde desde este programa se puede:

- Escribir y cargar código Compiscript
- Compilar el código
- Visualizar el árbol sintáctico
- Consultar todos los errores encontrados
- Revisar la tabla de símbolos
- Ejecutar las pruebas del proyecto

2. Objetivos del proyecto

2.1 Objetivo general

Desarrollar un analizador semántico para Compiscript que revise si el código cumple las reglas del lenguaje

Para ello, el sistema debe comprobar los tipos de datos, controlar dónde pueden utilizarse las variables y funciones, analizar el árbol sintáctico generado por ANTLR y guardar la información necesaria en una tabla de símbolos

2.2 Objetivos específicos

- Crear el analizador sintáctico de Compiscript utilizando ANTLR y la gramática ubicada en grammar/Compiscript.g4. Esta gramática puede volver a generarse ejecutando el comando `python build_grammar.py`.
- Construir el árbol sintáctico del programa y permitir que se visualice dentro del IDE. La visualización debe contar con zoom y también con una versión en texto para los casos en que Graphviz no esté disponible.
- Recorrer el árbol sintáctico mediante un Visitor de ANTLR para comprobar las reglas semánticas R1-R27. Estas reglas están organizadas en cuatro módulos o mixins, que después se unen en la clase principal SemanticVisitor.
- Crear una tabla de símbolos que organice las variables, funciones y clases según el entorno al que pertenecen: global, función, clase o bloque. Además, debe conservar un registro de todos los entornos creados durante el análisis.
- Mostrar todos los errores semánticos encontrados, indicando su categoría, línea y columna. El análisis no debe detenerse al encontrar el primer error, sino continuar para presentar una lista completa.
- Crear pruebas para verificar tanto programas correctos como programas con errores para cada regla semántica. Estas pruebas deben poder ejecutarse desde el IDE o mediante `pytest`.
- Desarrollar un IDE que permita escribir, cargar, analizar y compilar programas en Compiscript.

3. Fundamentos teóricos

Los algoritmos y estructuras implementados se apoyan en el Libro del Dragón, de Aho, Lam, Sethi y Ullman, *Compilers: Principles, Techniques, and Tools*, segunda edición, capítulos 2, 5, 6 y 7, y en la documentación oficial de ANTLR 4

3.1 Análisis semántico

Referencia teórica: el análisis semántico es la fase que sigue al análisis sintáctico, recibe el árbol sintáctico y verifica las propiedades del programa que una gramática libre de contexto no puede expresar, es decir que un nombre esté declarado antes de usarse, que los tipos de una operación sean compatibles y que una llamada respete la aridad de la función, además reúne la

información de tipos que las fases posteriores, generación de código intermedio y optimización, necesitarán, y la deposita en la tabla de símbolos

Implementación: el análisis se realiza en dos pasadas sobre el mismo árbol, la primera, el prepass, registra funciones globales y clases, y la segunda evalúa todas las reglas, la recuperación de errores es por acumulación y no por excepción, ya que cada regla que detecta un problema lo reporta y devuelve el tipo ERROR, que actúa como comodín para no encadenar errores derivados

```
# driver.py\ntree = parser.program()\n\nvisitor =
SemanticVisitor()\nejecutar_prepass(visitor, tree)  # pasada 1: funciones globales y
clases\nvisitor.visit(tree)                        # pasada 2: reglas R1-R27\n\nreturn Resultado(\n
tree=tree, parser=parser,\n    errores_sintacticos=errores_sintacticos,\n
errores_semanticos=visitor.reporter.all(),\n    table=visitor.table,\n)
```

3.2 Sistema de tipos

Referencia teórica, Dragón, capítulo 6: un sistema de tipos asigna una expresión de tipo a cada construcción del lenguaje y define cuándo dos tipos son compatibles, la conversión ampliadora, widening, preserva información, como al pasar de integer a float, y puede aplicarse de forma implícita, mientras que la reductora, narrowing, pierde información y exige una conversión explícita

Implementación: los tipos son objetos de una jerarquía, PrimitiveType, ArrayType, FunctionType y ClassType, y no cadenas de texto, la compatibilidad se concentra en una sola función, is_assignable(target, value), usada por igual en declaraciones, asignaciones, argumentos de llamada y retornos

```
# core/types.py\ndef is_assignable(target: Type, value: Type) -> bool:\n    if target ==\n    ERROR or value == ERROR:\n        return True\n        # DEC-4: no encadenar\n    errores\n    if target == value:\n        return True\n    if value == NULL:\n        # DEC-3: null solo a clase o arreglo\n        return isinstance(target, (ClassType,\n        ArrayType))\n    if target == FLOAT and value == INTEGER:\n        return True\n    # DEC-1: conversion ampliadora\n    if isinstance(target, ArrayType) and isinstance(value,\n    ArrayType):\n        return is_assignable(target.element_type, value.element_type)\n    if\n    isinstance(target, ClassType) and isinstance(value, ClassType):\n        cls = value\n    # subtipado por la cadena de herencia\n        while cls is not None:\n            if\n            cls.name == target.name:\n                return True\n                cls = cls.parent\n    return False
```

Decisiones de diseño documentadas como desviación del enunciado:

Decisión	Contenido	Justificación
DEC-1	Se agregó float a la gramática, integer se promueve a float	El enunciado pide aritmética con integer o float, y la gramática oficial no definía float
DEC-2	'+' está sobrecargado para string + string	Especificaciones.md usa concatenación, se acepta string+string y no string+T arbitrario
DEC-3	null solo es assignable a ClassType o ArrayType	No hay conversión válida de null a un primitivo, Dragón, sección 6.5.2
DEC-4	Existe un tipo ERROR que actúa como comodín	Recuperación de errores sin cascadas, un error se reporta una sola vez

Decisión	Contenido	Justificación
DEC-5	La expresión del switch puede ser de cualquier tipo, cada case debe coincidir con ella	Especificaciones.md muestra switch(x) con x de tipo integer, la bandera SWITCH_EXIGE_BOOLEAN permite invertirlo

3.3 Tabla de símbolos

Referencia teórica, Dragón, capítulos 2 y 7: la tabla de símbolos es la estructura que acompaña todas las fases del compilador, asocia cada identificador con sus atributos, tipo, clase de símbolo y ubicación, y responde dos preguntas, si un nombre puede declararse en el entorno actual y a qué declaración corresponde un uso

Implementación: un diccionario por entorno, con dos operaciones distinguidas según si la búsqueda sube o no por la cadena de padres

```
# core/symbols.py\nclass Scope:\n    def define(self, symbol: Symbol) -> bool:\n        if\n        symbol.name in self.symbols:\n            return False\n            # redeclaracion en ESTE\n        mismo scope\n        self.symbols[symbol.name] = symbol\n        return True\n        def\n        resolve(self, name: str) -> Symbol | None:\n            found = self.resolve_local(name)\n            if found is not None:\n                return found\n            if self.parent is not None:\n                return self.parent.resolve(name)\n            # sube por la cadena\n            return None
```

3.4 Ámbitos y entornos anidados

Referencia teórica: el ámbito de una declaración es la región del programa en la que esa declaración es visible, con reglas de ámbito estático la declaración que corresponde a un uso se determina inspeccionando el texto del programa, se busca en el bloque más interno y se avanza hacia afuera, una declaración interna que reutiliza un nombre externo lo oculta, shadowing, mientras dure su bloque

Implementación: pila de entornos vivos para el recorrido, cadena de punteros al padre para la visibilidad, e historial completo para el reporte final

```
# core/symbols.py\ndef enter_scope(self, kind: ScopeKind) -> Scope:\n    child =\n    Scope(kind=kind, parent=self.current())\n    self._stack.append(child)\n    # entornos\n    vivos\n    self._environments.append(child)\n    # historial (no se descarta al salir)\n    return child\n    def exit_scope(self) -> None:\n        if len(self._stack) <= 1:\n            raise\n            RuntimeError("No se puede salir del scope global")\n        self._stack.pop()
```

3.5 Listener o Visitor de ANTLR

Referencia teórica: ANTLR ofrece dos mecanismos para recorrer el árbol, el Listener es pasivo, ya que el caminante recorre el árbol y notifica la entrada y la salida de cada nodo sin que el código del usuario controle el orden ni devuelva valores, mientras que el Visitor es activo, cada método decide cuándo y en qué orden visita a sus hijos, y devuelve un valor

Se eligió el Visitor precisamente porque el análisis de tipos es una computación sintetizada de abajo hacia arriba, el tipo de una expresión se calcula a partir de los tipos de sus subexpresiones, y eso requiere que visitar un hijo devuelva un valor, con un Listener habría que acumular los tipos en una pila auxiliar, el control explícito del orden también es indispensable para abrir y cerrar entornos en el punto exacto, ya que visitFunctionDeclaration necesita declarar el nombre

de la función en el scope exterior, abrir el scope de función, declarar los parámetros y solo entonces visitar el cuerpo

```
# semantico/type_mixin.py -- sintesis de tipos de abajo hacia arriba\ndef
visitAdditiveExpr(self, ctx) -> Type:\n    operandos = ctx.multiplicativeExpr()\n    t =
self.get_type(operandos[0])\n    for i in range(1, len(operandos)):\n        operador =
self._operador_binario(ctx, i)\n        t = self._chequear_aritmetico(t,
self.get_type(operandos[i]), operador, ctx)\n    ctx.tipo = t        # se guarda en el nodo
para que el IDE lo consulte\n    return t
```

3.6 Verificación de tipos

Referencia teórica: un comprobador de tipos aplica reglas de inferencia de la forma si las subexpresiones tienen tipos T1 y T2 entonces la expresión tiene tipo T3, y reporta un error cuando ninguna regla aplica, las reglas se implementan como funciones que reciben los tipos de los operandos y devuelven el tipo del resultado

Implementación: cada familia de operadores tiene su chequeador, el aritmético admite el caso sobrecargado de '+' sobre cadenas y promueve al tipo mayor cuando ambos operandos son numéricos

```
# semantico/type_mixin.py\ndef _chequear_aritmetico(self, izq, der, operador, ctx) ->
Type:\n    if izq == ERROR or der == ERROR:\n        return ERROR\n    if operador ==
"+":\n        if is_numeric(izq) and is_numeric(der):\n            return max_tipo(izq,
der)        # integer + float -> float\n        if izq == STRING and der == STRING:\n            return STRING        # DEC-2: concatenacion\n        self.reporter.error(ctx,
"tipo", f"el operador '+' no admite {izq!r} y {der!r}")\n        return ERROR\n    if
is_numeric(izq) and is_numeric(der):\n        return max_tipo(izq, der)\n    self.reporter.error(ctx, "tipo",\n        f"el operador '{operador}' requiere operandos
numericos, "\n        f"se recibio {izq!r} y {der!r}")\n    return ERROR
```

La regla no multiplicar funciones, R26 del enunciado, no necesitó código dedicado, como una función se tipa con FunctionType y FunctionType no es numérico, el mismo chequeo aritmético la rechaza, en cambio $f(2) * g(3)$ es válido, porque ahí los operandos son los tipos de retorno

La validación de listas y de índices sigue el mismo patrón, el literal de arreglo unifica los tipos de sus elementos, promoviendo entre numéricos y reportando la categoría lista si son incompatibles, y el indexado exige que el objeto sea ArrayType y el índice integer, devolviendo element_type

3.7 Detección de código muerto

Referencia teórica: una sentencia es inalcanzable cuando ninguna ruta de ejecución puede llegar a ella, dentro de una secuencia lineal de sentencias cualquier instrucción que siga a un return, un break o un continue es inalcanzable, porque esos tres transfieren el control fuera del bloque de manera incondicional

Implementación: tras visitar normalmente la lista de sentencias de un bloque se recorre buscando el primer terminador, y si no es la última la siguiente se reporta como código muerto, se reporta solo la primera para no inundar el panel de errores con una fila por cada sentencia posterior, el mismo escaneo se aplica a los cuerpos de case y default del switch, que también son listas de sentencias


```
# semantico/function_mixin.py\ndef _escanear_codigo_muerto(self, statements) -> None:\n
    """R25: una sentencia despues de return/break/continue nunca se alcanza."""\n    for i,\n    stmt in enumerate(statements):\n        if self._es_terminador(stmt) and i + 1 <\n        len(statements):\n            self.reporter.error(statements[i + 1], "general",\n            "codigo muerto: esta sentencia nunca se ejecuta")\n            break\n\ndef\n_es_terminador(self, stmt) -> bool:\n    return (stmt.returnStatement() is not None\n    or stmt.breakStatement() is not None\n        or stmt.continueStatement() is not None)
```

3.8 Funciones anidadas, recursividad y closures

Referencia teórica, Dragón, capítulo 7: en un lenguaje con funciones anidadas una función puede referirse a variables declaradas en funciones que la encierran, esas variables no son ni locales ni globales, son no locales, la resolución estática de un nombre no local recorre la cadena de entornos de definición y no la de invocación, una closure es precisamente el par formado por el código de la función y el entorno de definición que captura

Implementación: la recursión se habilita declarando el nombre de la función en el scope exterior antes de visitar su propio cuerpo, de modo que la llamada recursiva ya lo resuelva, la recursión mutua entre funciones globales, que no se resolvería así porque la segunda función aún no existe cuando se visita la primera, la habilita el prepass, las closures y las funciones anidadas no requieren código adicional, ya que salen de la cadena de parent de los scopes

```
# semantico/function_mixin.py -- esquema de\nvisitFunctionDeclaration\nself.define_or_error(Symbol(name=fn_name, type=fn_type,\nkind="function"), ctx)\nself.enter_scope("function") # el nombre queda en el scope\nEXTERIOR\nfor parametro in parametros: # -> la llamada recursiva lo resuelve\n    self.define_or_error(Symbol(name=..., kind="param", ...),\n    parametro)\nself.function_stack.append((fn_name,\n    return_type))\nself._omitir_scope_de_bloque = True # el cuerpo no abre OTRO scope\n\nself.visit(ctx.block())\nself.function_stack.pop()\nself.exit_scope()
```

La validación de una llamada usa la firma almacenada en el símbolo, primero compara la aridad y, si coincide, verifica los tipos posicionales uno a uno

```
# semantico/function_mixin.py\ndef check_call_args(self, tipo_funcion: FunctionType,\nargs_ctx, ctx) -> None:\n    argumentos = args_ctx.expression()\n    if args_ctx is not None\n    else []\n    esperados = tipo_funcion.param_types\n    if len(argumentos) !=\n    len(esperados):\n        self.reporter.error(ctx, "funcion",\n        f"se esperaban\n        {len(esperados)} argumento(s), "\n        f"se recibieron {len(argumentos)}")\n    return\n    for esperado, arg in zip(esperados, argumentos):\n        tipo_arg =\n        self.get_type(arg)\n        if not is_assignable(esperado, tipo_arg):\n            self.reporter.error(arg, "funcion",\n            f"argumento de tipo {tipo_arg!r} no es\n            asignable "\n            f"al parametro de tipo {esperado!r}")
```

El mismo método lo reutiliza visitNewExpr para validar la invocación del constructor, R21, ya que un constructor es una functionDeclaration con nombre especial dentro de la clase, de modo que la verificación de argumentos es idéntica

4. Diseño de componentes

El sistema se organiza en capas que se comunican a través de un pipeline lineal de transformación de datos. El punto de entrada único del análisis es `driver.analizar()`, que consumen por igual el IDE, la línea de comandos y las pruebas automatizadas: ningún otro módulo instancia el lexer, el parser o el visitor por su cuenta.

Capa	Módulos principales	Responsabilidad
GUI / Orquestación	ide/app.py, main.py, driver.py	IDE, CLI y coordinación del pipeline completo
Análisis léxico y sintáctico	grammar/Compiscript.g4, build_grammar.py, generated/	Gramática ANTLR; lexer, parser y visitor generados
Contrato compartido	core/symbols.py, core/types.py, core/errors.py	Tabla de símbolos, sistema de tipos y reporte de errores
Análisis semántico	`semantico/` (4 mixins + prepass + visitor)	Reglas R1-R27 sobre el árbol sintáctico
Visualización	arbol/ast_graphviz.py	ParseTree a PNG en memoria con Graphviz
Pruebas	pruebas/, tests/	Batería .cps y pruebas unitarias con pytest

Pipeline completo:

```
archivo.cps -> CompiscriptLexer -> CommonTokenStream -> CompiscriptParser
               -> ParseTree (program)
               -> ejecutar_prepass() (pasada 1: funciones globales y clases)
               -> SemanticVisitor.visit() (pasada 2: reglas R1-R27)
               -> Resultado(errores_sintacticos, errores_semanticos, table, tree)
               -> IDE / CLI / runner de pruebas

grammar/Compiscript.g4 -> build_grammar.py -> generated/CompiscriptLexer.py
                                           generated/CompiscriptParser.py
                                           generated/CompiscriptVisitor.py
```

4.1 Módulo léxico

El módulo léxico se genera a partir de la gramática `grammar/Compiscript.g4` y se materializa en `generated/CompiscriptLexer.py`, su tarea es transformar el texto del archivo `.cps` en una secuencia de tokens que el parser pueda consumir

Entre los tokens reconocidos se encuentran:

- Palabras reservadas del lenguaje, como `let`, `const`, `function`, `class`, `if`, `while`, `for`, `return`, `print`
- Identificadores de variables, funciones, clases y atributos
- Literales enteros, flotantes, de cadena y booleanos, junto con el literal `null`
- Operadores aritméticos, relacionales, lógicos y de asignación
- Signos de agrupación y puntuación, como paréntesis, llaves, corchetes y punto y coma

Los espacios en blanco, los saltos de línea y los comentarios de una o varias líneas se descartan en esta etapa, cuando el lexer encuentra un carácter que no pertenece al alfabeto del lenguaje reporta un error léxico con su línea y columna, y el proceso continúa para no interrumpir el análisis en el primer hallazgo

4.2 Módulo gramatical

La gramática de Compiscript se define en `grammar/Compiscript.g4` con la notación de ANTLR, allí se describen tanto las reglas del lexer como las del parser, el equipo amplió la gramática original para incorporar el tipo `float` y su valor literal, de modo que el lenguaje admite números decimales

La gramática se regenera con el comando `python build_grammar.py`, que produce el lexer, el parser y el visitor dentro del directorio `generated/`, este esquema mantiene una sola fuente de verdad para el lenguaje, ya que cualquier cambio en la sintaxis se realiza únicamente en el archivo `.g4` y luego se propaga al código generado

4.3 Módulo sintáctico

El parser generado en `generated/CompiscriptParser.py` toma la secuencia de tokens producida por el lexer y construye el árbol sintáctico del programa, la regla inicial es `program`, y a partir de ella se derivan las declaraciones, las sentencias y las expresiones del lenguaje

El árbol resultante es la estructura sobre la que trabajan las etapas posteriores, por lo que el parser no realiza ninguna comprobación de tipos ni de ámbitos, únicamente valida la forma del programa

Los errores sintácticos se recolectan en una lista propia y se muestran por separado de los errores semánticos, cuando el programa contiene errores de sintaxis el árbol puede quedar incompleto, en ese caso el sistema informa los hallazgos al usuario y evita continuar con un análisis que produciría mensajes poco confiables

4.4 Módulo semántico

El análisis semántico se encuentra en el paquete `semantico/` y está dividido en mixins que agrupan reglas afines, todos se combinan en la clase principal `SemanticVisitor`, que hereda del visitor generado por ANTLR y recorre el árbol sintáctico

El módulo trabaja en dos pasadas:

- La función `ejecutar_prepass()` registra las funciones globales y las clases antes del análisis principal, con esto una función puede invocarse o una clase puede heredarse aunque su declaración aparezca más adelante en el archivo
- El recorrido de `SemanticVisitor.visit()` revisa cada instrucción y aplica las reglas R1-R27, entre ellas la compatibilidad de tipos, la existencia previa de los identificadores, la aridad y el tipo de los argumentos en las llamadas, el uso correcto de clases, objetos y listas, y la validez de las sentencias de control de flujo

Cada regla incumplida genera un objeto de error con su categoría, mensaje, línea y columna, el recorrido no se detiene al encontrar el primer problema, de manera que el usuario recibe el listado completo en una sola compilación

4.5 Módulo de tabla de símbolos

La tabla de símbolos se implementa en `core/symbols.py`, junto con `core/types.py` para el sistema de tipos y `core/errors.py` para el reporte de errores, estos tres archivos forman el contrato compartido entre el análisis semántico y las capas de presentación

La tabla almacena cada declaración con la información necesaria para validarla:

- Nombre del símbolo y su categoría, es decir variable, constante, parámetro, función, clase o atributo
- Tipo declarado o inferido, incluyendo los tipos `integer`, `float`, `string`, `boolean`, `null`, las listas y los tipos definidos por las clases
- Entorno al que pertenece, que puede ser global, de función, de clase o de bloque
- Posición en el código fuente, con línea y columna, para ubicar la declaración original

Los entornos se organizan de forma anidada, cada vez que el análisis entra en una función, una clase o un bloque se crea un entorno hijo que apunta a su entorno padre, la búsqueda de un identificador comienza en el entorno actual y asciende hasta el global, con esto se resuelven el sombreado de nombres y el acceso a variables externas dentro de funciones anidadas y closures

El módulo conserva además un registro de todos los entornos creados durante el análisis, lo que permite mostrarlos completos en el IDE aunque ya se hayan cerrado

4.6 Módulo de visualización del árbol sintáctico

La visualización se implementa en `arbol/ast_graphviz.py`, este módulo recorre el `ParseTree` entregado por el parser y lo convierte en una imagen PNG generada en memoria con `Graphviz`, sin escribir archivos temporales en el disco

Los nodos internos muestran el nombre de la regla gramatical y las hojas muestran el texto del token correspondiente, la imagen se despliega dentro del IDE con control de zoom para poder revisar árboles extensos

Cuando `Graphviz` no está instalado en el sistema el módulo entrega una representación en texto con sangrías, de esta manera la herramienta sigue siendo utilizable en cualquier entorno

4.7 Interfaz gráfica o IDE

El IDE se desarrolló con `CustomTkinter` y se encuentra en `ide/app.py`, mientras que `main.py` ofrece el acceso por línea de comandos, ambos consumen el mismo punto de entrada `driver.analizar()`, por lo que la interfaz no duplica ninguna lógica del análisis

Las funciones disponibles para el usuario son:

- Escribir código `Compiscript` en el editor o cargar un archivo `.cps` existente
- Compilar el programa y obtener el resultado del análisis en una sola acción

- Visualizar el árbol sintáctico con zoom o en su versión de texto
- Consultar el reporte de errores sintácticos y semánticos, con categoría, línea y columna
- Revisar la tabla de símbolos con todos los entornos generados durante el análisis
- Ejecutar la batería de pruebas del proyecto desde la propia interfaz

5. Diseño de estructuras de datos

El análisis se apoya en un conjunto reducido de estructuras que comparten todos los módulos, cada una tiene una responsabilidad única y un módulo propietario:

Estructura	Módulo	Campos principales	Propósito
Symbol	core/symbols.py	name, type, kind, initialized, line, column	Entrada de la tabla de símbolos
Scope	core/symbols.py	kind, parent, symbols{}	Un entorno, la cadena de parent da la visibilidad
SymbolTable	core/symbols.py	_global, _stack, environments	Pila de entornos vivos y historial completo
Type (jerarquía)	core/types.py	PrimitiveType, ArrayType, FunctionType, ClassType	Sistema de tipos del lenguaje
SemanticError	core/errors.py	line, column, category, message	Un error semántico localizado
ErrorReporter	core/errors.py	errors[]	Acumula errores sin detener el análisis
Resultado	driver.py	tree, parser, errores_sintacticos, errores_semanticos, table	Salida única del pipeline hacia el IDE y la CLI
ResultadoPrueba	pruebas/runner.py	archivo, es_valido, caso, categorias_esperadas, paso, razon	Veredicto de un archivo .cps de la batería

5.1 Estructura de la tabla de símbolos

La tabla mantiene dos colecciones a la vez, y esa es su decisión de diseño central:

```
SymbolTable\n  _stack          [global] -> [function] -> [block]          # entornos VIVOS:
busqueda actual\n  _environments [global, function, class, block, ...] # HISTORIAL: salida
para el IDE\n\nScope(kind, parent, symbols{})\n  define(symbol)      inserta solo en ESTE
scope, False si ya existe\n  resolve(name)        busca aqui y, si no, sube por parent
(visibilidad)\n  resolve_local(name) solo este scope (redeclaracion / shadowing)
```

Al ejecutar `exit_scope()` el ámbito deja de ser visible para `resolve`, de modo que no hay fuga de variables de un bloque hacia su padre, pero el entorno permanece en `_environments`

El enunciado pide como salida el estado de la tabla por cada entorno, no solo por los que siguen abiertos al terminar el programa, conservar el historial es lo que hace posible ese reporte

El método `snapshot()` devuelve los entornos vivos en un momento dado, `environments()` devuelve todos los creados y `format_environments()` los formatea como texto para el panel del IDE:

```
[0] global: var x: integer, function foo: (integer) -> integer\n[1] function: param n:  
integer\n[2] block: var tmp: integer
```

5.2 Representación de símbolos

Cada declaración se guarda en una instancia de `Symbol`, definida como `dataclass`:

```
@dataclass\nclass Symbol:\n    name: str\n    type: Type          # core.types.Type\n    kind: str          # "var" | "const" | "param" | "function" | "class"\n    initialized: bool = True\n    line: int = 0\n    column: int = 0
```

Los valores válidos de `kind` están congelados como constantes del módulo, `SYMBOL_KINDS` y `SCOPE_KINDS`, para que los cuatro mixins usen exactamente los mismos strings

El campo `type` guarda una instancia de la jerarquía de tipos y no un string, de manera que una función se almacena como `FunctionType` y una clase como `ClassType`, esa decisión hace que reglas como la prohibición de multiplicar funciones, R26, no necesiten código propio, ya que el chequeo aritmético rechaza cualquier operando que no sea numérico y un `FunctionType` no lo es

La jerarquía de tipos se define en `core/types.py`:

Tipo	Definición	Notas de diseño
<code>PrimitiveType</code>	<code>PrimitiveKind: INTEGER, FLOAT, STRING, BOOLEAN, NULL, VOID, ERROR</code>	Instancias únicas y frozen, <code>VOID</code> es el retorno de una función sin anotación
<code>ArrayType</code>	<code>element_type: Type</code>	Anidable, <code>integer[][]</code> es <code>ArrayType(ArrayType(INTEGER))</code>
<code>FunctionType</code>	<code>param_types: tuple, return_type: Type</code>	Permite validar aridad y tipos posicionales de una llamada
<code>ClassType</code>	<code>name, parent, fields {}, methods {}</code>	<code>eq=False</code> y comparación por nombre, una clase autoreferente colgaría el <code>eq</code> generado

El tipo `ERROR`, identificado como DEC-4, es un mecanismo de recuperación, cuando una regla ya reportó un error devuelve `ERROR`, y `is_assignable` trata `ERROR` como comodín, así un único error de tipo no produce una cascada de errores derivados en las expresiones que lo contienen

5.3 Manejo de entornos y ámbitos

Existen cuatro clases de entorno, y cada una se abre en un punto preciso del recorrido:

Entorno	Se abre en	Contiene
<code>global</code>	Lo crea <code>SymbolTable</code> al construirse	Variables, constantes, funciones y clases de nivel superior
<code>function</code>	<code>visitFunctionDeclaration</code>	Parámetros con <code>kind="param"</code> y el cuerpo de la función
<code>class</code>	<code>visitClassDeclaration</code>	Campos y métodos de la clase
<code>block</code>	<code>visitBlock</code>	Variables locales del bloque, incluido el cuerpo de <code>if</code> , <code>while</code> y <code>for</code>

Un detalle fino es que el bloque de una función no abre un segundo scope, si lo hiciera, un parámetro y una variable local con el mismo nombre no colisionarían y la redeclaración pasaría inadvertida, por eso visitFunctionDeclaration activa la bandera `_omitir_scope_de_bloque` antes de visitar el cuerpo, de modo que el scope de función y el del bloque sean el mismo

La resolución de nombres no usa la pila de llamadas sino la cadena de parent, es decir, el entorno donde el nombre fue definido, eso es exactamente lo que hace que las closures funcionen, una función anidada ve las variables del entorno en el que fue declarada y no las del entorno desde el que se la invoca

5.4 Estructura de los errores semánticos

Cada hallazgo del análisis se representa con una instancia de `SemanticError`:

```
@dataclass\nnclass SemanticError:\n    line: int\n    column: int\n    category: str\n    message: str\n    def __str__(self):\n        return f"[{self.category}] linea {self.line}:{self.column} - {self.message}"
```

Las siete categorías están alineadas con la salida que pide el enunciado y se validan con un `assert` en el momento de reportar, de modo que una categoría mal escrita falle de inmediato y no en silencio:

Categoría	Qué agrupa	Ejemplo de mensaje
tipo	Sistema de tipos	el operador '*' requiere operandos numericos, se recibio string y integer
ambito	Resolución y declaración de nombres	'y' no esta declarado en este ambito
funcion	Llamadas y retornos	se esperaban 2 argumento(s), se recibieron 1
control_flujo	if, while, for, switch, break, continue y return	'break' solo es valido dentro de un bucle
clase	Clases, herencia, this y miembros	la clase padre 'Animal' no esta declarada
lista	Arreglos e índices	el indice debe ser integer, se recibio string
general	Código muerto y errores sintácticos	codigo muerto: esta sentencia nunca se ejecuta

El método `ErrorReporter.error()` acepta indistintamente un `ParserRuleContext`, del que usa `.start`, un `TerminalNode` de ANTLR, del que usa `.symbol`, o un `Token` directo, normalizar los tres casos en un solo punto evitó un `AttributeError` que aparecía en decenas de sitios donde el error se reporta sobre un identificador suelto en lugar de sobre una regla completa

Ningún módulo lanza excepciones ni imprime, todos reportan al mismo `ErrorReporter`, por eso una sola compilación muestra la lista completa de errores del archivo en vez de detenerse en el primero

6. Interfaz gráfica — Vistas

El IDE se lanza con `python main.py`, sin argumentos, la ventana principal mide 1100x750 px y se compone de una barra de herramientas con los botones Abrir, Guardar, Guardar como, Compilar (F5) y Correr batería, el editor con numeración de líneas, un panel de pestañas con las cuatro vistas descritas a continuación y una barra de estado que resume el resultado de la última compilación en verde cuando no hay hallazgos o en rojo indicando el número de errores

6.1 Editor de código

El editor es un área de texto con deshacer y rehacer habilitados y una columna lateral de números de línea que se recalcula al escribir, al desplazarse y al redimensionar la ventana, los botones Abrir y Guardar como usan los diálogos nativos del sistema filtrados por la extensión `.cps`

El botón Compilar, o la tecla F5, envía el contenido actual del editor a `driver.analizar()` y no el archivo en disco, de modo que se puede compilar código sin guardarlo, y con una sola acción se repintan las cuatro pestañas

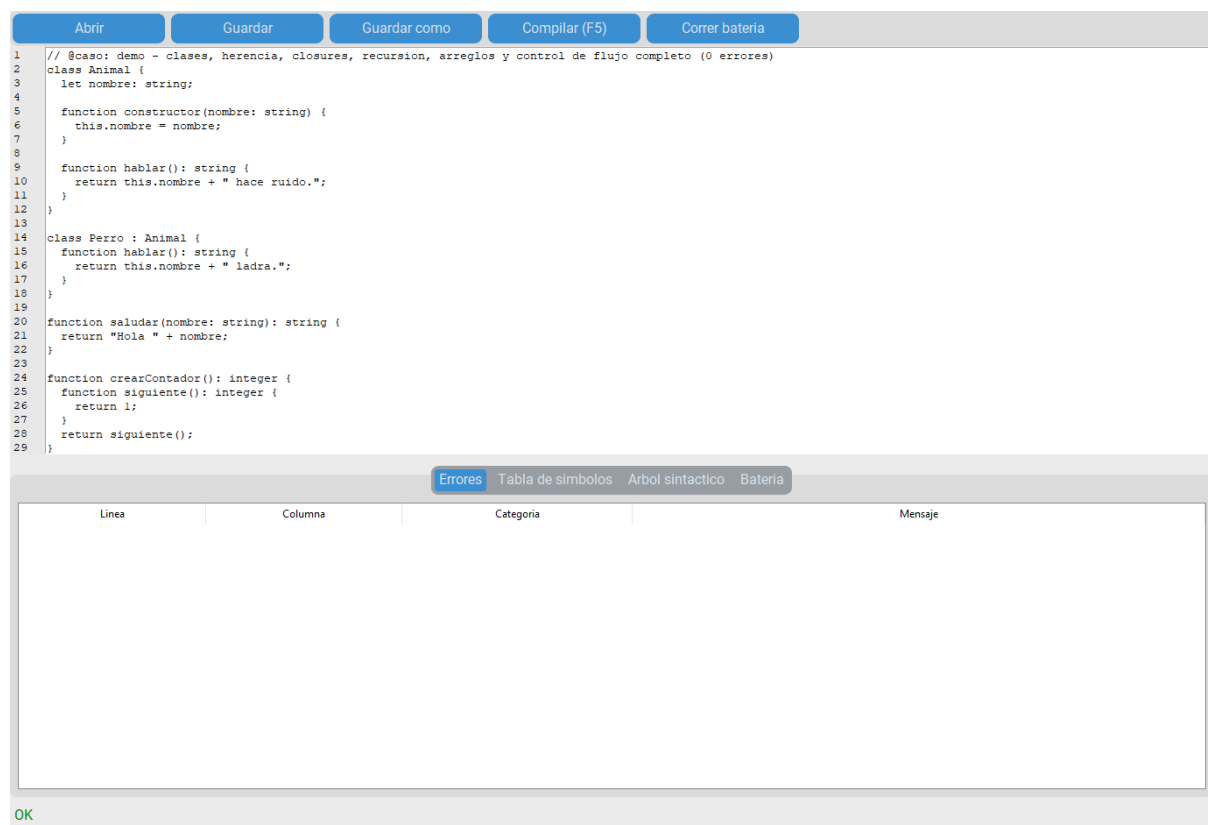


Figura 1. Vista del editor con numeración de líneas y la barra de herramientas

6.2 Visualización del árbol sintáctico

La pestaña Árbol sintáctico dibuja el ParseTree completo del programa sobre un Canvas con barras de desplazamiento vertical y horizontal, y dos botones de zoom, Acercar y Alejar, con un factor de 1.25 por paso, que reescalan la imagen sin volver a invocar a Graphviz

Los nodos de regla aparecen como elipses azules con el nombre de la regla de la gramática y las hojas como cajas amarillas con el texto del token, cuando el binario dot no está instalado el panel muestra automáticamente la representación en texto del árbol, de manera que el IDE nunca queda sin salida

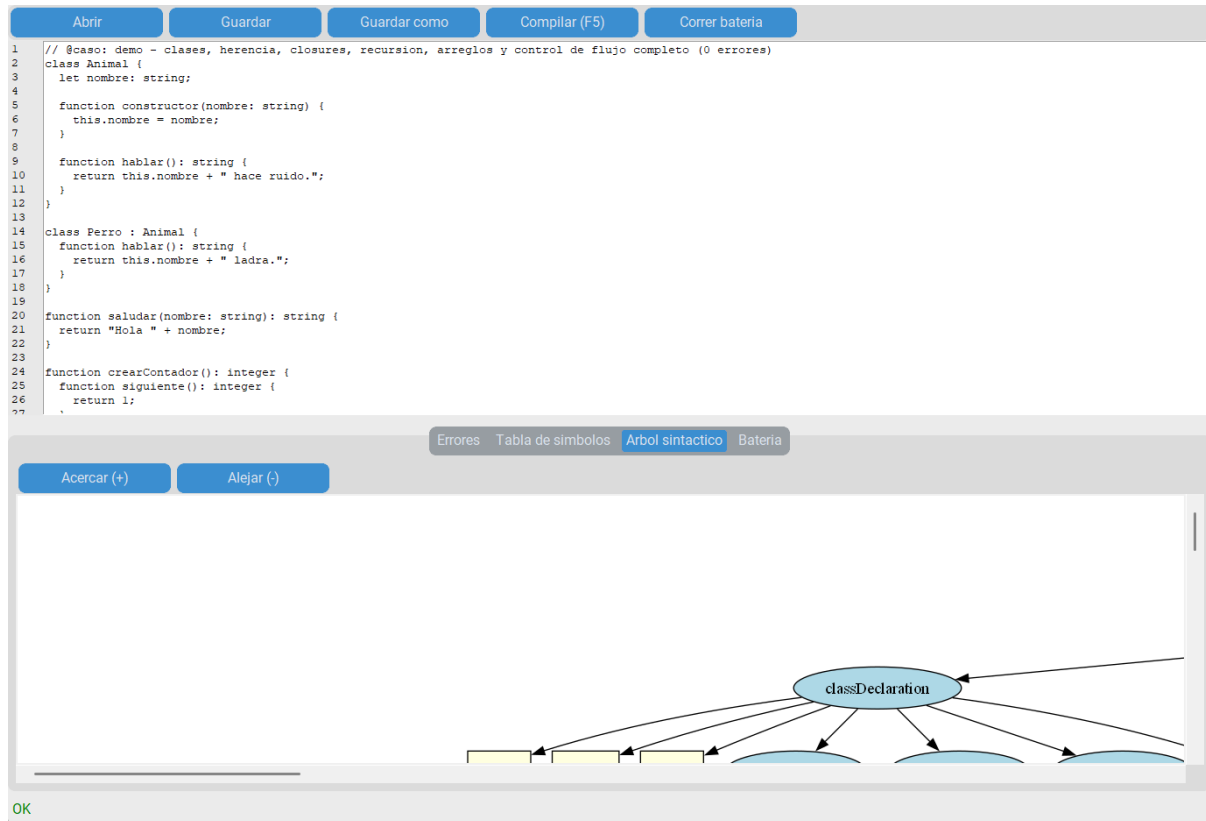


Figura 2. Árbol sintáctico renderizado con Graphviz dentro del IDE

6.3 Reporte de errores

La pestaña Errores presenta una tabla construida con ttk.Treeview y cuatro columnas, línea, columna, categoría y mensaje, se llena con Resultado.errores, que ya viene ordenado por posición y mezcla los errores sintácticos detectados por ANTLR con los semánticos reportados por el visitor

Un doble clic sobre cualquier fila mueve el cursor del editor a esa línea y la desplaza a la vista, para corregirla sin buscarla a mano

Abrir

Guardar

Guardar como

Compilar (F5)

Correr batería

```

1 // @caso: demo - exactamente un error de cada una de las 7 categorías
2 // @error: tipo
3 // @error: ambito
4 // @error: funcion
5 // @error: control_flujo
6 // @error: clase
7 // @error: lista
8 // @error: general
9
10 // tipo: asignar string a integer
11 let a: integer = "texto";
12
13 // ambito: variable no declarada
14 print(bNoDeclarada);
15
16 // funcion: numero de argumentos incorrecto
17 function suma(x: integer, y: integer): integer { return x + y; }
18 let s: integer = suma(1);
19
20 // control_flujo: break fuera de un bucle
21 break;
22
23 // clase: atributo inexistente
24 class Animal { let nombre: string; }
25 let animal: Animal = new Animal();
26 print(animal.edad);
27
28 // lista: indice no entero
29 let lista: integer[] = [1, 2, 3];

```

Errores

Tabla de símbolos

Arbol sintactico

Batería

Linea	Columna	Categoría	Mensaje
11	0	tipo	no se puede asignar string a 'a' de tipo integer
14	6	ambito	'bNoDeclarada' no esta declarado en este ambito
18	21	funcion	se esperaban 2 argumento(s), se recibieron 1
21	0	control_flujo	'break' solo es valido dentro de un bucle
26	12	clase	'Animal' no tiene el miembro 'edad'
30	11	lista	el indice debe ser integer, se recibio string
35	2	general	codigo muerto: esta sentencia nunca se ejecuta

7 errores

Figura 3. Reporte de errores con categoría y ubicación, el doble clic salta a la línea

6.4 Visualización de la tabla de símbolos

La pestaña Tabla de símbolos muestra el resultado de `Resultado.table.format_environments()`, es decir una línea por cada entorno creado durante el análisis, incluidos los ya cerrados, con su índice, su clase, que puede ser global, function, class o block, y la lista de símbolos que contiene, cada uno con su kind y su tipo, esa es exactamente la salida del estado de la tabla de símbolos por cada entorno que pide el enunciado



Figura 4. Estado de la tabla de símbolos por entorno

Una cuarta pestaña, Batería, ejecuta pruebas/runner.ejecutar_bateria() con el botón Correr batería y reporta el conteo de casos que pasaron junto con el detalle de cada fallo, sin salir del IDE

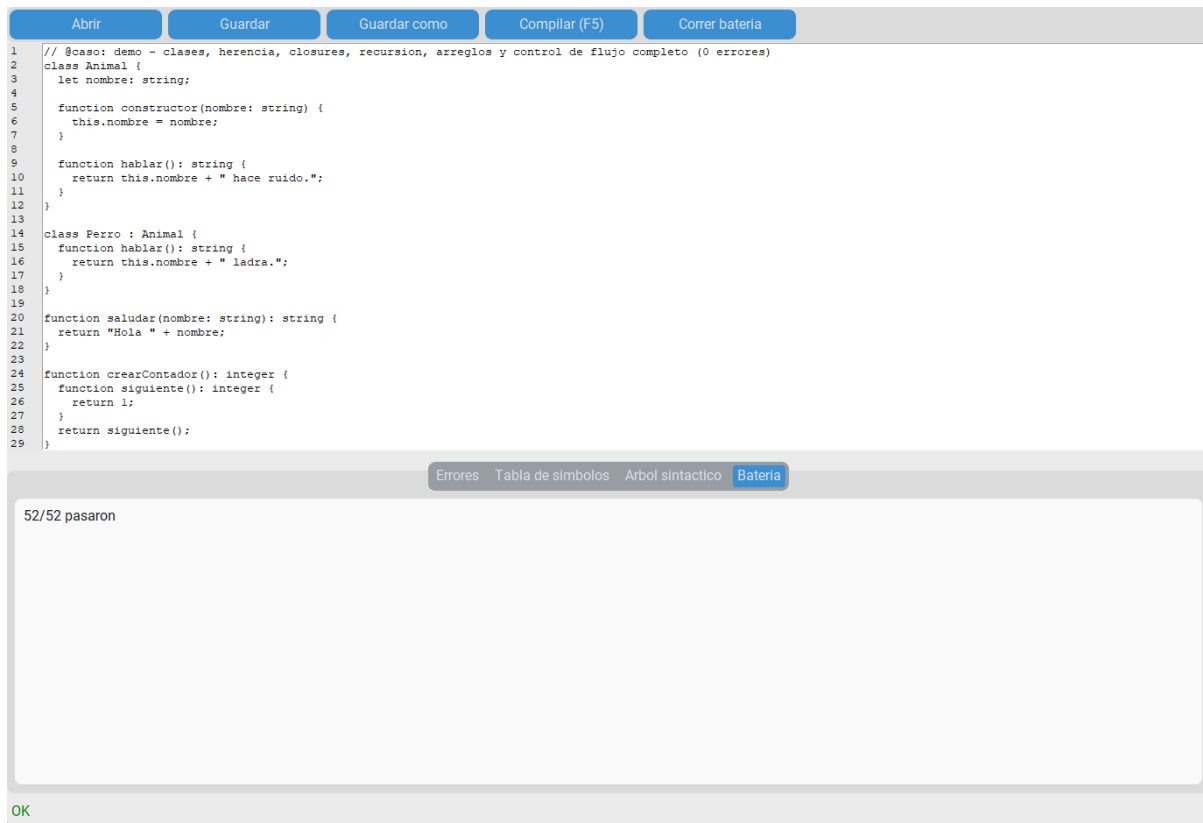


Figura 5. Ejecución de la batería de pruebas desde el IDE

7. Conjuntos de prueba

La batería vive en `pruebas/<categoria>/{validos,invalidos}/*.cps` y consta de 52 archivos .cps, 27 válidos y 25 inválidos, que cubren las 27 reglas semánticas del enunciado, cada archivo lleva una cabecera obligatoria que el runner interpreta:

```
// @caso: R12 argumentos de menos en una llamada\n// @error: funcion          <- solo en
los invalidos, una o mas lineas
```

- En validos/ el runner exige 0 errores, tanto sintácticos como semánticos
- En invalidos/ exige al menos 1 error y que todas las categorías declaradas en @error: aparezcan entre las reportadas, esa segunda condición evita el falso positivo de un archivo que falla pero por otra razón, ya que un archivo que debería dar un error de tipo y da uno de ámbito no cuenta como aprobado

Distribución por categoría:

Categoría	Válidos	Inválidos	Reglas del enunciado
tipos	6	6	R1-R5 (aritmética, lógicos, comparaciones, asignación, const) y DEC-3 (null)
ámbito	4	4	R7-R10 (resolución, no declarada, redeclaración, bloques)
funciones	5	3	R12-R16 (argumentos, retorno, recursión, closures, redeclaración)

Categoría	Válidos	Inválidos	Reglas del enunciado
control_flujo	3	4	R17-R19 (condiciones booleanas, break/continue, return)
clases	3	3	R20-R22 (miembros, constructor, this)
listas	2	2	R6, R23, R24 (tipos de elementos e índices)
generales	3	3	R25-R27 (código muerto, sentido semántico, duplicados)
tabla_simbolos	1	0	R11 (nuevo entorno por función, clase y bloque)
TOTAL	27	25	52 archivos .cps

Tres reglas, R11 sobre el nuevo entorno, R14 sobre la recursión y R15 sobre las closures, solo tienen archivo válido, ya que son reglas estructurales sin una condición de error natural, una closure que resuelve bien no tiene un caso inválido propio, sus fallas caen en R7-R9 y ahí están cubiertas

Además de la batería, tests/ contiene pruebas unitarias por módulo, test_symbols, test_tipos, test_funciones, test_clases y test_driver, junto con el guardarril de colisión de mixins, test_mixins, en total 153 pruebas pasan con pytest, contando la parametrización de los 52 archivos de la batería

Por último, pruebas/demo/ contiene dos programas de presentación que quedan fuera de la batería automática a propósito, programa_completo.cps, con clases, herencia, closures, recursión, arreglos y todo el control de flujo y 0 errores, y programa_errores.cps, que produce exactamente un error de cada una de las siete categorías

7.1 Sistema de tipos

Archivo	Caso
tipos/validos/aritmetica_numerica.cps	R1: operadores + - * / con operandos integer y float
tipos/invalidos/aritmetica_tipos_incompatibles.cps	R1: operando string en una multiplicación
tipos/validos/logicos.cps	R2: operadores &&, y ! sobre operandos boolean
tipos/invalidos/logicos_no_booleanos.cps	R2: operando integer en un &&
tipos/validos/comparaciones.cps	R3: ==, !=, <, <=, >, >= entre tipos compatibles
tipos/invalidos/comparaciones_incompatibles.cps	R3: comparación entre string e integer
tipos/validos/asignacion.cps	R4: valor del mismo tipo que la variable declarada
tipos/invalidos/asignacion_incompatible.cps	R4: string asignado a una variable integer
tipos/validos/const.cps	R5: constante inicializada en su declaración
tipos/invalidos/const_reasignada.cps	R5: reasignación de una constante
tipos/validos/null_dec3.cps	DEC-3: null asignado a un tipo de clase o arreglo
tipos/invalidos/null_dec3.cps	DEC-3: null asignado a un primitivo

7.2 Manejo de ámbitos

Archivo	Caso
ambito/validos/resolucion.cps	R7: resolución de un nombre local y de uno global
ambito/invalidos/resolucion_antes_de_declarar.cps	R7: uso de una variable antes de declararla
ambito/validos/no_declarada.cps	R8: todas las variables usadas están declaradas
ambito/invalidos/no_declarada.cps	R8: uso de un identificador inexistente
ambito/validos/redeclaracion.cps	R9: mismo nombre en ámbitos distintos, shadowing

Archivo	Caso
ambito/invalidos/redeclaracion.cps	R9: dos declaraciones del mismo nombre en un ámbito
ambito/validos/bloques.cps	R10: acceso correcto desde un bloque anidado
ambito/invalidos/bloques.cps	R10: uso de una variable local fuera de su bloque

7.3 Funciones y procedimientos

Archivo	Caso
funciones/validos/argumentos.cps	R12: llamada con la aridad y los tipos correctos
funciones/invalidos/argumentos_numero.cps	R12: llamada con menos argumentos de los declarados
funciones/validos/retorno.cps	R13: return del tipo declarado
funciones/invalidos/retorno.cps	R13: return de un tipo distinto al declarado
funciones/validos/recursion.cps	R14: función recursiva, factorial
funciones/validos/closure.cps	R15: función anidada que captura una variable externa
funciones/validos/sin_redeclaracion.cps	R16: funciones con nombres distintos
funciones/invalidos/redeclaracion.cps	R16: dos funciones con el mismo nombre

7.4 Control de flujo

Archivo	Caso
control flujo/validos/cond booleanas.cps	R17: condiciones boolean en if, while, do-while y for
control flujo/invalidos/cond no booleana.cps	R17: condición de tipo integer
control flujo/validos/break continue.cps	R18: break y continue dentro de un bucle
control flujo/invalidos/break fuera de bucle.cps	R18: break fuera de todo bucle
control flujo/invalidos/continue fuera de bucle.cps	R18: continue fuera de todo bucle
control flujo/validos/return en funcion.cps	R19: return dentro del cuerpo de una función
control flujo/invalidos/return fuera de funcion.cps	R19: return en el nivel global

7.5 Clases y objetos

Archivo	Caso
clases/validos/miembro.cps	R20: acceso a un atributo y a un método existentes
clases/invalidos/miembro_inexistente.cps	R20: acceso a un miembro que la clase no define
clases/validos/constructor.cps	R21: new con los argumentos que el constructor declara
clases/invalidos/constructor_aridad.cps	R21: new con un número incorrecto de argumentos
clases/validos/this.cps	R22: this dentro de un método de la clase
clases/invalidos/this_fuera_de_clase.cps	R22: this fuera de cualquier clase

7.6 Listas y estructuras de datos

Archivo	Caso
listas/validos/elementos_numericos.cps	R6 y R23: literal de arreglo con elementos del mismo tipo
listas/invalidos/elementos_incompatibles.cps	R6 y R23: literal con elementos de tipos distintos
listas/validos/indice.cps	R24: indexado con una expresión integer
listas/invalidos/indice_no_entero.cps	R24: índice de tipo no entero

7.7 Validaciones generales

Archivo	Caso
generales/validos/sin_codigo_muerto.cps	R25: return como última sentencia del bloque
generales/invalidos/codigo_muerto.cps	R25: sentencia después de un return

Archivo	Caso
generales/validos/multiplicar_funciones_ok.cps	R26: multiplicación entre los resultados de dos llamadas
generales/invalidos/multiplicar_funciones.cps	R26: multiplicación de las funciones mismas, sin invocarlas
generales/validos/sin_duplicados.cps	R27: variables y parámetros con nombres únicos
generales/invalidos/duplicados.cps	R27: parámetro duplicado en la misma función
tabla_simbolos/validos/entornos_anidados.cps	R11: un entorno por cada función, clase y bloque

8. Restricciones cumplidas

Requisito del enunciado	Verificación	Estado
Analizador sintáctico con ANTLR sobre la gramática de Compiscript	grammar/Compiscript.g4 + build_grammar.py generan lexer, parser y visitor en generated/	CUMPLIDA
Árbol sintáctico con representación visual	arbol/ast_graphviz.py, Graphviz en memoria, y pestaña Árbol sintáctico con zoom y fallback en texto	CUMPLIDA
Recorrido del árbol con Listener o Visitor de ANTLR	SemanticVisitor combina cuatro mixins sobre CompiscriptVisitor	CUMPLIDA
Sistema de tipos, aritmética, lógicos, comparaciones, asignaciones, const y listas	semantico/type_mixin.py, reglas R1-R6, R23 y R24	CUMPLIDA
Manejo de ámbito, resolución, no declarada, redeclaración, bloques y un entorno por función, clase y bloque	core/symbols.py + semantico/function_mixin.py, reglas R7-R11	CUMPLIDA
Funciones, argumentos, retorno, recursión, anidamiento, closures y redeclaración	semantico/function_mixin.py + semantico/prepass.py, reglas R12-R16	CUMPLIDA
Control de flujo, condiciones booleanas, break, continue y return	semantico/function_mixin.py, reglas R17-R19	CUMPLIDA
Clases y objetos, miembros con '.', constructor y this	semantico/class_mixin.py + ClassType.lookup_member, reglas R20-R22	CUMPLIDA
Validaciones generales, código muerto, sentido semántico y duplicados	semantico/function_mixin.py y core/symbols.py, reglas R25-R27	CUMPLIDA
Tabla de símbolos con entornos anidados, global, función, clase y bloque	core/symbols.py, pila + cadena de parent + historial de entornos	CUMPLIDA
Reporte de errores con tipo y ubicación	core/errors.py, siete categorías, línea y columna, pestaña Errores del IDE	CUMPLIDA
Estado de la tabla de símbolos por cada entorno	SymbolTable.environments() y format_environments(), pestaña Tabla de símbolos	CUMPLIDA
Batería de pruebas con casos exitosos y fallidos por regla	52 archivos .cps, 27 válidos y 25 inválidos, más 153 pruebas pytest	CUMPLIDA
Batería ejecutable al momento de la evaluación	Botón Correr batería en el IDE y python -m pytest tests/ -q	CUMPLIDA
IDE funcional para escribir, cargar y compilar Compiscript	ide/app.py, CustomTkinter, editor con números de línea, F5, cuatro pestañas y barra de estado	CUMPLIDA
Commits individuales por integrante	Historial de git con autoría diferenciada por módulo, sección 10	CUMPLIDA

Requisito del enunciado	Verificación	Estado
Documentación de arquitectura y de ejecución	README.md, docs/ARCHITECTURE.md, docs/COBERTURA.md, docs/Tabla de Simbolos.md	CUMPLIDA

9. Manual de instalación y ejecución

9.1 Requisitos y dependencias

Requisito	Versión o nota
Python	3.11 o superior
Java (JDK)	En el PATH, lo usa antlr4-tools para generar el parser
Graphviz	Instalado en el sistema, con el binario dot en el PATH
antlr4-python3-runtime	4.13.2, fijada, debe coincidir con el generador
antlr4-tools	Generador de ANTLR
customtkinter	Interfaz gráfica del IDE
pillow	Renderizado de la imagen del árbol en el Canvas
graphviz	Binding de Python para producir el PNG
pytest	Ejecución de las pruebas

Instalación de las dependencias de Python:

```
git clone https://github.com/lin231135/CC3032-Proyecto.git\ncd CC3032-Proyecto\npip install -r requirements.txt
```

Generación del parser: el código que produce ANTLR no se versiona, ya que generated/ está en .gitignore, así que hay que generarlo después de clonar el repositorio y cada vez que cambie la gramática:

```
python build_grammar.py
```

El script deja CompiscriptLexer.py, CompiscriptParser.py, CompiscriptVisitor.py y CompiscriptListener.py en generated/, usando la misma versión de ANTLR que el runtime instalado, si Graphviz no está disponible el resto del sistema funciona igual, solo el panel del árbol cae al fallback de texto

9.2 Ejecución del IDE

```
python main.py
```

Sin argumentos, main.py lanza el IDE, el flujo de trabajo típico es el siguiente:

1. Abrir un archivo .cps con el botón Abrir, o escribir el código directamente en el editor
2. Pulsar Compilar o F5, el contenido actual del editor, guardado o no, se analiza
3. Revisar la pestaña Errores, cada fila trae línea, columna, categoría y mensaje, y un doble clic lleva el cursor a esa línea del editor
4. Revisar la pestaña Tabla de símbolos para ver el estado de cada entorno creado durante el análisis
5. Revisar la pestaña Árbol sintáctico y usar Acercar o Alejar para inspeccionar el árbol
6. Guardar los cambios con Guardar o Guardar como

9.3 Compilación de archivos .cps

El mismo análisis está disponible desde la línea de comandos, pasando la ruta del archivo:

```
python main.py archivo.cps
```

Imprime la lista de errores, o el mensaje OK: 0 errores, seguida de la tabla de símbolos por entorno, el código de salida es 0 si el archivo compiló sin errores y 1 en caso contrario, lo que permite encadenarlo en un script, un ejemplo de salida es el siguiente:

```
$ python main.py pruebas/demo/programa_errores.cps\n[tipo] linea 4:12 - el operador '*'
requiere operandos numericos, se recibio string y integer\n[ambito] linea 7:2 - 'y' no esta
declarado en este ambito\n[funcion] linea 12:9 - se esperaban 2 argumento(s), se recibieron
1\n...\n\n[0] global: var x: integer, function foo: (integer) -> integer\n[1] function:
param n: integer\n[2] block: var tmp: integer
```

También puede invocarse el análisis desde código Python, que es lo mismo que hacen el IDE y las pruebas:

```
from driver import analizar_archivo\n\nresultado =
analizar_archivo("pruebas/demo/programa_completo.cps")\n\nprint(resultado.ok)
# True si no hubo errores\n\nfor error in resultado.errores:      # ordenados por (linea,
columna)\n    print(error)\n\nprint(resultado.table.format_environments())
```

9.4 Ejecución de las pruebas

Suite completa, es decir la batería .cps parametrizada más las pruebas unitarias por módulo:

```
python -m pytest tests/ -q
```

Solo la batería de archivos .cps:

```
python -m pytest tests/test_bateria.py -q
```

O directamente el runner, que es lo mismo que ejecuta el botón Correr batería del IDE:

```
python -m pruebas.runner\n\n# o desde codigo:\n\nfrom pruebas.runner import
ejecutar_bateria\n\nfor resultado in ejecutar_bateria():\n    print(resultado,
resultado.razon)
```

Para agregar un caso nuevo basta con crear un archivo en pruebas/<categoria>/validos/ o en pruebas/<categoria>/invalidos/ con la cabecera // @caso: y, si es inválido, una o más líneas // @error: <categoria>, el runner lo descubre solo, por lo que no hay que registrarlo en ningún lado

10. Participación y distribución del trabajo

El trabajo se repartió en cuatro frentes con una interfaz común, el contrato de core/, congelado el primer día, y un mixin propio por persona, de modo que nadie editara el mismo archivo que otro y los commits quedaran individualizados, como exige el enunciado, la distribución siguiente se corresponde con la autoría registrada en el historial de git del repositorio

Integrante	Módulo	Aporte
Jennifer Toxcon (21276)	Tabla de símbolos y contrato de core/	Implementó Symbol, Scope, SymbolTable y ScopeHelpers con entornos anidados e historial de entornos, el ErrorReporter con las

Integrante	Módulo	Aporte
		siete categorías, el sistema de tipos base de core/types.py, la extensión de la gramática con float, DEC-1, y la documentación docs/Tabla de Simbolos.md
Gadiel Ocaña (231270)	Sistema de tipos, ámbito, funciones y control de flujo	Implementó type_mixin.py, con tipos de expresiones, declaraciones, asignaciones, listas e índices y el fold de leftHandSide, y function_mixin.py, con bloques, funciones, llamadas, condiciones booleanas, break, continue, return y código muerto, además del driver de análisis y la CLI
Javier Linares (231135)	Clases y objetos, integración y batería de pruebas	Definió la estructura inicial del repositorio y el contrato de core/, implementó class_mixin.py, con clases, herencia, this, new y acceso a miembros, el prepass de dos pasadas y la corrección del orden del MRO de SemanticVisitor, y la batería completa de archivos .cps con su runner
Cindy Gualim (21226)	IDE, visualización del árbol y documentación	Desarrolló el IDE con CustomTkinter, editor con números de línea, compilación con F5 y las pestañas de errores, tabla de símbolos, árbol y batería, y el renderizado del árbol con Graphviz, corrigió la asignación de null a clases y arreglos, DEC-3, y el reporte duplicado de tipos desconocidos, y elaboró la matriz de cobertura y la documentación del proyecto

La integración siguió un orden fijo, primero el contrato de core/, luego los mixins de tipos y de funciones en paralelo, después el de clases, y al final el prepass y el reordenamiento del MRO, que es donde los cuatro frentes empezaron a funcionar de punta a punta, el frente del IDE avanzó en paralelo desde el primer día, porque solo depende del objeto Resultado y no de la lógica de ninguna regla

11. Conclusiones

- El analizador semántico cubre las 27 reglas del enunciado y las cuatro salidas exigidas, árbol visual, reporte de errores con tipo y ubicación, estado de la tabla por entorno y batería ejecutable, verificadas una por una en la matriz de cobertura del repositorio
- Separar el recorrido en cuatro mixins permitió que cuatro personas trabajaran en paralelo sobre una sola clase de ANTLR sin conflictos de merge, el costo de esa decisión fue descubrir que el orden del MRO importa, ya que un stub de un mixin puede tapar en silencio la implementación real de otro, la prueba que verifica que ningún par de mixins define el mismo visit* convirtió ese incidente en un guardarril permanente

- Concentrar toda la compatibilidad de tipos en una sola función, `is_assignable()`, evitó duplicar la regla en declaraciones, asignaciones, argumentos y retornos, tipar también las funciones y las clases hizo que reglas como no multiplicar funciones salieran gratis del chequeo aritmético, sin código adicional
- El tipo `ERROR` como comodín resultó clave para la usabilidad del IDE, sin él un único error de tipo generaba una cascada de errores derivados que ocultaba la causa real entre decenas de filas
- Conservar el historial de entornos, y no solo los vivos, fue lo que permitió cumplir el requisito de mostrar el estado de la tabla por cada entorno, con una pila simple los scopes de bloque y de función habrían desaparecido antes de poder reportarlos
- La pasada previa resolvió de raíz la dependencia del orden de declaración, la recursión mutua entre funciones globales y la herencia hacia adelante dejaron de ser casos especiales y pasaron a funcionar sin código adicional en la pasada principal
- La convención de la batería, exigir que las categorías declaradas en la cabecera aparezcan entre las reportadas, evita el falso positivo más común al probar un compilador, dar por buena una prueba que efectivamente falló pero por una razón distinta de la que se quería verificar
- La tabla de símbolos quedó diseñada como servicio compartido e independiente del recorrido, por lo que puede sostener sin cambios las fases posteriores del compilador, generación de código intermedio y asignación de memoria

12. Enlace al repositorio de GitHub

El código fuente completo del proyecto, la batería de pruebas y la documentación técnica se encuentran en el siguiente repositorio:



<https://github.com/lin231135/CC3032-Proyecto>

Documentación complementaria dentro del repositorio:

Archivo	Contenido
README.md	Requisitos, instalación, ejecución y estructura del repositorio
docs/ARCHITECTURE.md	Arquitectura, patrón de mixins y repartición del trabajo
docs/COBERTURA.md	Matriz regla del enunciado -> código -> prueba -> estado
docs/Tabla de Simbolos.md	Diseño y contrato de la tabla de símbolos
docs/Especificaciones.md	Especificación del lenguaje Compiscript
pruebas/README.md	Convención de la batería de archivos .cps

13. Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). Compilers: Principles, Techniques, and Tools (2a ed.). Addison-Wesley. [Libro del Dragón]
- [2] Parr, T. (2013). The Definitive ANTLR 4 Reference (2a ed.). Pragmatic Bookshelf.
- [3] ANTLR (2024). ANother Tool for Language Recognition. <https://wwwantlr.org/>
- [4] CustomTkinter (2024). Modern and customizable python UI-library based on Tkinter. <https://github.com/TomSchimansky/CustomTkinter>
- [5] Graphviz (2024). Graph visualization software. <https://graphviz.org/>
- [6] Pillow (2024). The Python Imaging Library fork. <https://python-pillow.org/>
- [7] pytest (2024). The pytest framework. <https://docs.pytest.org/>